

Week 8 - Image Segmentation 2 (Clustering and Region-Based Segmentation, Batch Processing, Evaluation)

[3/Apr/2026 Dr Min Wang]

Overview

Image segmentation partitions an image into meaningful regions corresponding to objects or structures. In this lab, we continue play with different classical image segmentation methods. We will implement:

1. Region Growing segmentation (Region-based)
2. K-Means segmentation (clustering-based)
3. Mean Shift segmentation (clustering-based)

In addition, we will learn how to:

1. Process a folder of images automatically
2. Evaluate segmentation results using quantitative measures:
 - Intersection over Union (IoU)
 - Dice Coefficient
3. Visualise segmentation results of a few image examples

Objectives

- Understand different segmentation principles
- Implement and compare different methods, analyse parameters
- Automatic processing a folder of images
- Calculate evaluation metrics

Requirements

You need the following libraries, you should have been familiar with them by now, but install them if you haven't done so.

pip install numpy opencv-python matplotlib scikit-learn

Part 1: Region Growing Segmentation

Algorithm Steps:

1. Select seed pixel
2. Compare neighboring pixels
3. Add pixel if similarity condition satisfied
4. Repeat until no more pixels can be added

```
In [68]: # Setup: Import all required libraries.
# - OpenCV: image processing
# - NumPy: numerical operations
# - Matplotlib: visualization

import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams['figure.figsize'] = (4,4)
```

```

In [131... # =====
# 1. Load One Example Image
# =====

# We start with a single image to understand how segmentation works.
image_path = "Abyssinian_1.jpg" # This is the image
annotation_path = "Abyssinian_1.png" # The corresponding annotation (mask), we will use it as ground t

image = cv2.imread(image_path)
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

plt.figure(1)
plt.title("Original Image")
plt.imshow(image)
plt.axis('off')

plt.figure(2)
plt.title("Grey Image")
plt.imshow(gray)
plt.axis('off')

```

```

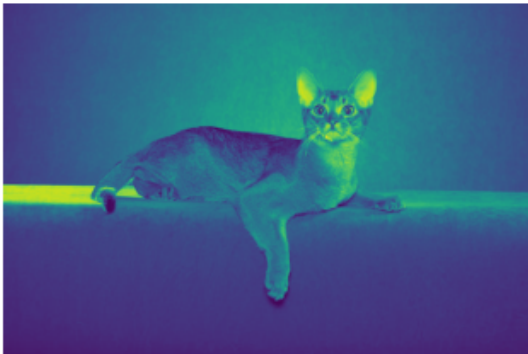
Out[131... (np.float64(-0.5), np.float64(599.5), np.float64(399.5), np.float64(-0.5))

```

Original Image



Grey Image



```

In [132... # =====
# 2. Region Growing (Demo on Single Image)
# =====

# Region growing starts from a seed pixel and expands to neighbors if they are similar in intensity.
# Key parameters:
# - seed: starting pixel
# - threshold: how similar pixels must be

# Simple implementation inline (for understanding)
def region_growing_demo(image, seed, threshold=15):
    h, w = image.shape
    segmented = np.zeros((h, w), dtype=np.uint8)
    visited = np.zeros((h, w), dtype=bool)

    seed_value = image[seed]
    stack = [seed]

    while stack:
        x, y = stack.pop()

        if visited[x, y]:
            continue

```

```

        visited[x, y] = True

    if abs(int(image[x, y]) - int(seed_value)) < threshold:
        segmented[x, y] = 255

    # 4-connected neighbors
    neighbors = [(x-1,y),(x+1,y),(x,y-1),(x,y+1)]

    for nx, ny in neighbors:
        if 0 <= nx < h and 0 <= ny < w:
            stack.append((nx, ny))

    return segmented

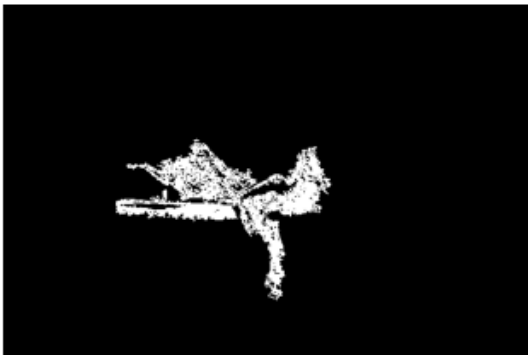
# Try different seeds and thresholds to observe behavior
h, w = gray.shape
seed = (h//2, w//2) # let's start with the central pixel
region_result = region_growing_demo(gray, seed, threshold=15)

plt.title("Region Growing Result")
plt.imshow(region_result, cmap='gray')
plt.axis('off')

```

Out[132... (np.float64(-0.5), np.float64(599.5), np.float64(399.5), np.float64(-0.5))

Region Growing Result



Part 2: K-Means Segmentation

Algorithm Steps:

1. Define features
2. Specify K - number of clusters
3. Clustering

```

In [133... # =====
# 2. K-Means Segmentation (Demo on Single Image)
# =====

# K-Means clusters pixels into K groups based on similarity.
# We must specify the number of clusters (K).
# Here, we use color features only: [R, G, B]

# Prepare data
Z = image.reshape((-1,3))
Z = np.float32(Z)

# Define number of clusters
K = 2

# Apply K-Means
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 10, 1.0)
_, labels, centers = cv2.kmeans(Z, K, None, criteria, 10, cv2.KMEANS_RANDOM_CENTERS)

# Convert back to image
centers = np.uint8(centers)
segmented_kmeans = centers[labels.flatten()]
segmented_kmeans = segmented_kmeans.reshape((image.shape))

plt.title("K-Means Segmentation Result")

```

```
plt.imshow(segmented_kmeans)
plt.axis('off')
```

```
Out[133... (np.float64(-0.5), np.float64(599.5), np.float64(399.5), np.float64(-0.5))
```

K-Means Segmentation Result



Part 3: Mean Shift Segmentation

Mean Shift is a non-parametric clustering algorithm. It does NOT require specifying the number of clusters (unlike K-means).

Core idea:

- Each pixel is treated as a point in a feature space
- We iteratively shift each point toward regions of higher density
- Points that converge to the same peak belong to the same cluster

Pros:

- Automatically finds number of clusters
- Works well for complex, non-linear structures

Cons:

- Computationally expensive
- Sensitive to bandwidth selection

```
In [73]: # =====
# 3. Mean Shift (Demo on Single Image)
# =====

# In image segmentation, the feature space typically includes:
#   - Color (R, G, B)
#   - Spatial position (x, y)

# Key parameters:
# - bandwidth: controls the size of the neighborhood (very important!)
#   * small bandwidth → many small segments (over-segmentation)
#   * large bandwidth → fewer large regions (under-segmentation)

from sklearn.cluster import MeanShift

h, w, _ = image.shape

# Build feature vectors: [R, G, B, x, y]
X = []
for i in range(h):
    for j in range(w):
        r, g, b = image[i, j]
        X.append([r, g, b, i/10, j/10]) # scales down the spatial coordinates, so they are comparable to color

X = np.array(X)

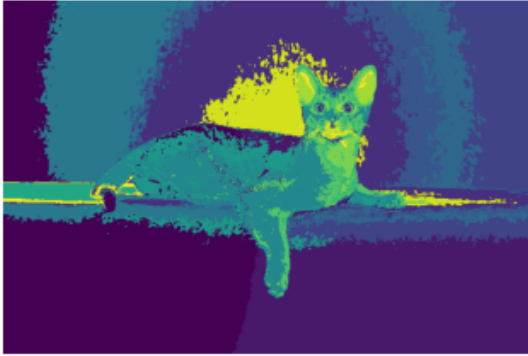
# Apply Mean Shift clustering
ms = MeanShift(bandwidth=20, bin_seeding=True)
labels = ms.fit_predict(X)

segmented = labels.reshape((h, w))
```

```
plt.title("Mean Shift Result (Cluster Map)")
plt.imshow(segmented)
plt.axis('off')
```

Out[73]: (np.float64(-0.5), np.float64(599.5), np.float64(399.5), np.float64(-0.5))

Mean Shift Result (Cluster Map)



Define Functions for Batch Processing

Now that we understand the methods, we formalise them into reusable functions.

```
In [120]: # =====
# Define reusable functions
# =====

# Region growing method
def region_growing(image, seed, threshold=15):
    h, w = image.shape
    segmented = np.zeros((h, w), dtype=np.uint8)
    visited = np.zeros((h, w), dtype=bool)

    seed_value = image[seed]
    stack = [seed]

    while stack:
        x, y = stack.pop()

        if visited[x, y]:
            continue

        visited[x, y] = True

        if abs(int(image[x, y]) - int(seed_value)) < threshold:
            segmented[x, y] = 1

        neighbors = [(x-1,y),(x+1,y),(x,y-1),(x,y+1)]

        for nx, ny in neighbors:
            if 0 <= nx < h and 0 <= ny < w:
                stack.append((nx, ny))

    return segmented

# K-Means method
def kmeans_segmentation(image, K=3):
    Z = image.reshape((-1,3))
    Z = np.float32(Z)

    criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 10, 1.0)
    _, labels, centers = cv2.kmeans(Z, K, None, criteria, 10, cv2.KMEANS_RANDOM_CENTERS)

    centers = np.uint8(centers)
    segmented = centers[labels.flatten()]
    segmented = segmented.reshape((image.shape))

    return segmented

# Mean Shift method
def mean_shift_segmentation(image, spatial_radius=10, color_radius=20):
    h, w, c = image.shape
```

```

X = []
for i in range(h):
    for j in range(w):
        r, g, b = image[i, j]
        X.append([r, g, b, i/spatial_radius, j/spatial_radius])

X = np.array(X)

ms = MeanShift(bandwidth=color_radius, bin_seeding=True)
labels = ms.fit_predict(X)

segmented = labels.reshape((h, w))
return segmented

```

Define Evaluation Metrics

To evaluate the segmentation results, we compare the predicted segmentation map against the ground truth annotation/mask.

Each pixel is classified as **foreground (pet)** or **background**, so the evaluation becomes a pixel-wise comparison:

	Ground Truth: Foreground	Ground Truth: Background
Predicted Foreground	True Positive (TP)	False Positive (FP)
Predicted Background	False Negative (FN)	True Negative (TN)

We can then define the following two evaluation metrics:

1. Intersection over Union (IoU)

IoU measures how well the predicted segmentation overlaps with the ground truth. It is defined as the ratio between the area of overlap and the area of union between the predicted mask and the ground truth mask:

$$IoU = \frac{TP}{TP + FP + FN}$$

IoU provides a value between **0 and 1**, where:

- **1** indicates perfect segmentation
- **0** indicates no overlap

A higher IoU indicates better segmentation performance.

2. Dice Coefficient (F1 Score)

The Dice Coefficient is similar to IoU but slightly more forgiving. It measures the similarity between the predicted segmentation and the ground truth by considering their overlap.

It is calculated as:

$$Dice = \frac{2TP}{2TP + FP + FN}$$

A higher Dice score indicates better performance, with:

- **1** indicating a perfect match
- **0** indicates no overlap

```

In [121... # =====
# Evaluation metrics
# =====

# Define IoU
def compute_iou(gt, pred):
    intersection = np.logical_and(gt, pred).sum()
    union = np.logical_or(gt, pred).sum()
    return intersection / (union + 1e-6)

# Define Dice

```

```
def compute_dice(gt, pred):
    intersection = np.logical_and(gt, pred).sum()
    return (2. * intersection) / (gt.sum() + pred.sum() + 1e-6)
```

In [122... *# Let's see an example here: suppose we have the following prediction and the ground truth*

```
gt_example = np.array([
    [1, 1, 0, 0],
    [1, 1, 0, 0],
    [0, 0, 0, 0],
    [0, 0, 0, 0]
])

pred_example = np.array([
    [1, 0, 0, 0],
    [1, 1, 0, 0],
    [0, 0, 0, 0],
    [0, 0, 0, 0]
])

plt.figure(figsize=(8,3))

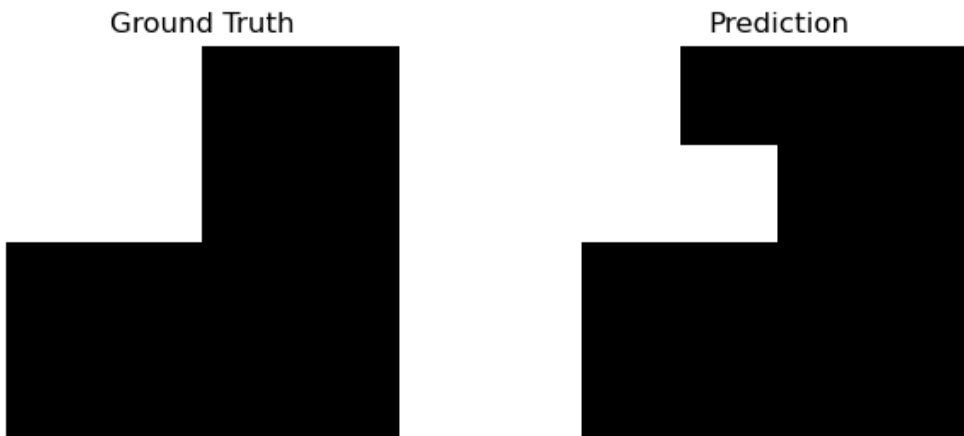
plt.subplot(1,2,1)
plt.title("Ground Truth")
plt.imshow(gt_example, cmap='gray')
plt.axis('off')

plt.subplot(1,2,2)
plt.title("Prediction")
plt.imshow(pred_example, cmap='gray')
plt.axis('off')

plt.show()

# Compute metrics for the example
iou_example = compute_iou(gt_example, pred_example)
dice_example = compute_dice(gt_example, pred_example)

print(f"Example IoU: {iou_example:.3f}")
print(f"Example Dice: {dice_example:.3f}")
```



Example IoU: 0.750
Example Dice: 0.857

Batch Processing

Finally, let's implement batch processing to process multiple images automatically.

Folder structure: dataset/

- "images" contains all the images
- "masks" contains the corresponding annotation (segmentation mask) for the images

In [129... *# =====*
Batch Processing
=====

We now process a folder of images automatically.

```

# Folder structure:
# dataset/
#   |— images/
#   |— masks/

# The dataset labels:
# 1 = Foreground (pet)
# 2 = Background
# 3 = Not classified (ignore)

# We will convert GT masks to binary (1 = foreground, 0 = else) and
# ignore pixels labeled 3 when computing IoU and Dice.

import os
from tqdm import tqdm

# Create folder to save predictions
save_folder = "dataset/predictions"
os.makedirs(save_folder, exist_ok=True)

results = []

def process_and_save(image_folder, mask_folder, method='region'):
    results = []

    for filename in tqdm(os.listdir(image_folder)):
        # Image file is JPG
        img_path = os.path.join(image_folder, filename)
        # Corresponding mask file is PNG (replace extension)
        mask_name = os.path.splitext(filename)[0] + '.png'
        mask_path = os.path.join(mask_folder, mask_name)

        image = cv2.imread(img_path)
        image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        gray = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2GRAY)

        # Load GT mask
        gt = cv2.imread(mask_path, 0) # single-channel

        # Convert GT to binary mask (foreground = 1)
        gt_binary = np.where(gt == 1, 1, 0)
        # Valid mask (exclude pixels labeled 3)
        valid_mask = gt != 3

        # Apply segmentation method
        if method == 'region':
            h, w = gray.shape
            center_seed = (h//2, w//2)
            pred_mask = region_growing(gray, seed=center_seed, threshold=15)
        elif method == 'mean_shift':
            pred_seg = mean_shift_segmentation(image_rgb)
            # Use center pixel cluster for foreground
            h, w, _ = image_rgb.shape
            pred_mask = (pred_seg == pred_seg[h//2, w//2]).astype(np.uint8)
        elif method == 'kmeans':
            pred_seg = kmeans_segmentation(image_rgb, K=3)
            # Use center pixel cluster for foreground
            h, w, _ = image_rgb.shape
            center_cluster = pred_seg[h//2, w//2, 0] # R channel
            pred_mask = (pred_seg[:, :, 0] == center_cluster).astype(np.uint8)
        else:
            raise ValueError("Unknown method")

        # Save segmentation result as PNG
        save_path = os.path.join(save_folder, f"{os.path.splitext(filename)[0]}_{method}.png")
        cv2.imwrite(save_path, (pred_mask*255).astype(np.uint8))

        # Compute metrics using valid pixels only
        iou = compute_iou(gt_binary[valid_mask], pred_mask[valid_mask])
        dice = compute_dice(gt_binary[valid_mask], pred_mask[valid_mask])

        results.append((filename, iou, dice, save_path))
    return results

# Example usage

```



```

image_folder = "dataset/images"
mask_folder = "dataset/masks"
results = process_and_save(image_folder, mask_folder, method='mean_shift')

# Print summary
for name, iou, dice, save_path in results:
    print(f"{name}: IoU={iou:.3f}, Dice={dice:.3f}, Saved: {save_path}")

```

```
100%|██████████| 5/5 [02:18<00:00, 27.77s/it]
```

```

Abyssinian_1.jpg: IoU=0.454, Dice=0.624, Saved: dataset/predictions/Abyssinian_1_mean_shift.png
Abyssinian_3.jpg: IoU=0.369, Dice=0.540, Saved: dataset/predictions/Abyssinian_3_mean_shift.png
Abyssinian_2.jpg: IoU=0.002, Dice=0.004, Saved: dataset/predictions/Abyssinian_2_mean_shift.png
Abyssinian_5.jpg: IoU=0.389, Dice=0.560, Saved: dataset/predictions/Abyssinian_5_mean_shift.png
Abyssinian_4.jpg: IoU=0.142, Dice=0.248, Saved: dataset/predictions/Abyssinian_4_mean_shift.png

```

Visualise Best and Worst Cases

Finally, we can visualise the good and bad segmentation cases and analyse the results.

```

In [130... # =====
# Visualize Best and Worst Cases
# =====

# Sort results by IoU (you can also sort by Dice if preferred)
sorted_results = sorted(results, key=lambda x: x[1], reverse=True)

best_cases = sorted_results[:2]
worst_cases = sorted_results[-2:]

def visualize_cases(cases, title):
    print(f"{title}")
    for filename, iou, dice, save_path in cases:
        img_path = os.path.join(image_folder, filename)
        mask_name = os.path.splitext(filename)[0] + '.png'
        mask_path = os.path.join(mask_folder, mask_name)

        image = cv2.imread(img_path)
        image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        gt = cv2.imread(mask_path, 0)
        gt_binary = np.where(gt == 1, 1, 0)

        pred_mask = cv2.imread(save_path, 0) // 255

        plt.figure(figsize=(12,4))
        plt.subplot(1,3,1)
        plt.title("Original Image")
        plt.imshow(image_rgb)
        plt.axis('off')

        plt.subplot(1,3,2)
        plt.title("Ground Truth")
        plt.imshow(gt_binary, cmap='gray')
        plt.axis('off')

        plt.subplot(1,3,3)
        plt.title(f"Prediction IoU={iou:.2f}, Dice={dice:.2f}")
        plt.imshow(pred_mask, cmap='gray')
        plt.axis('off')

        plt.show()

# Visualize
visualize_cases(best_cases, "Top 2 Best Segmentation Results")
visualize_cases(worst_cases, "Top 2 Worst Segmentation Results")

```

Top 2 Best Segmentation Results

Original Image



Ground Truth



Prediction IoU=0.45, Dice=0.62



Original Image



Ground Truth



Prediction IoU=0.39, Dice=0.56



Top 2 Worst Segmentation Results

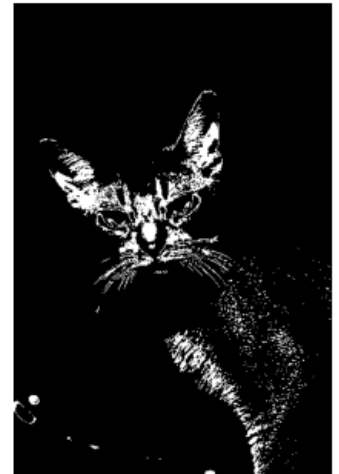
Original Image



Ground Truth



Prediction IoU=0.14, Dice=0.25



Original Image



Ground Truth



Prediction IoU=0.00, Dice=0.00



Discussion

- What is your observation?
- What is the impact of the parameters?
- Why do you think are the reasons for the failure cases?
- How would you like to improve performance?

In []: